■ DevOps Daily Practice — Single Consolidated Study Book

Azure CLI + Terraform + Git + Docker + AKS/Kubernetes + VM + Management Groups + Interview Scenarios + SPN/WIF

All supplied practice notes are consolidated into one reading flow. Credential-like values from the source notes are redacted.

1. Azure CLI + Daily Azure Operations

This chapter combines the Azure command notes and the Azure-related troubleshooting entries.

CORE COMMANDS

```
az login
az account list --output table
az account show --query id --output tsv
az account set --subscription <subscription-id>
az group show --name <RG-NAME>
az group show --name <RG-NAME> --query location -o table
az resource list -o table
az vm list -o table

Storage / Terraform backend practice:
az group create --name aksrg --location East US
az storage account create --name askustg --resource-group aksrg --location Eastus --sku standard_LRS --kind StorageV2
az storage container create --name tfstate --account-name askustg --auth-mode login

Resource discovery:
az network vnet subnet list --resource-group <RG> --vnet-name <VNET> -o table
az network nic list --resource-group <RG> -o table

VM:
az vm start --name <VMName> --resource-group <RGName>
```

The source notes also contain examples for selecting a subscription when a subscription-related portal operation fails.

2. Terraform State, Import, Drift & Resource Lifecycle

KEY IDEA

```
Terraform state tells Terraform which real Azure resources it is tracking. The source notes repeatedly use:
terraform state list
terraform state rm <resource-address>
terraform import <resource-address> <resource-id>
terraform plan
terraform apply
terraform destroy
```

IMPORT EXISTING RESOURCE

When Azure already contains a resource and Terraform says it already exists, the notes use this flow:

1. Identify the Terraform resource address.
2. Build the Azure resource ID.
3. Run terraform import.
4. Verify with terraform state list.
5. Run terraform plan.

```
Generic:
terraform import <resource_address> <resource_id>
```

Examples from the notes include:
- azurerm_storage_account
- azurerm_resource_group
- module.frontend_subnet.azurerm_subnet.subnet
- module.Backend_subnet.azurerm_subnet.subnet
- module.public_ip_name.azurerm_public_ip.example

SUBNET ALREADY EXISTS

The notes show a module calling:
module "subnetcalling" { ... }

When frontend_subnet already exists, the documented solution is to import it into state.

SUBNET CIDR OVERLAP

Error:
NetcfgSubnetRangesOverlap

```
Troubleshooting:
az network vnet subnet list --resource-group lb_rg --vnet-name lb_vnet -o table
```

Then inspect AddressPrefix and choose a non-overlapping range in Terraform.

SUBNET DELETE

```
az network vnet subnet delete --name frontend_subnet --resource-group lb_rg --vnet-name lb_vnet

If the subnet is in use, the notes say to check NICs first:
az network nic list --resource-group lb_rg -o table
```

STATE REMOVE

```
If a resource was manually deleted or should no longer be tracked:
terraform state rm <resource-address>

Then:
terraform plan
```

The notes also mention terraform refresh as a state-related command.

DESTRUCTIVE OPERATIONS

```
terraform destroy
terraform destroy -target <resource>
```

The source notes repeatedly emphasize avoiding direct Portal deletion for Terraform-managed resources.

3. Terraform Authentication, SPN & Secrets

The source Git notes show a provider with subscription_id and explain that local Terraform may rely on Azure CLI authentication.

```
Local CLI flow from the notes:
az logout
az login --scope https://management.azure.com/.default

For automation, the notes show a Service Principal approach:
az ad sp create-for-rbac --name terraform-sp --role Contributor --scopes /subscriptions/<subscription-id>
```

The notes then show an AzureRM provider using:
subscription_id
client_id
client_secret
tenant_id

SECURITY PRACTICE FROM THE NOTES

The supplied material includes an actual secret example. It has been redacted from this book.

The notes explain:
- Do not hardcode secrets in provider.tf.
- Use variables.
- Keep sensitive terraform.tfvars out of Git.
- Use environment variables or CI/CD secret stores.
- If a secret was committed, removing it only from the current file is not enough; history may still contain it.
- git filter-repo was used in the notes for history cleanup.
- Temporary GitHub secret-unblock is described as not recommended.

The source notes also describe Azure DevOps / pipeline use of Service Principal authentication.

4. Azure Management Groups — Interview + Design Notes

The spreadsheet contains a complete Management Group interview section.

KEY RESOURCE NAMES

azurerm_management_group
azurerm_management_group_subscription_association
azurerm_role_assignment
azurerm_policy_assignment

CORE USE CASES

- Hierarchy
- Governance
- Access control
- Policies

- Enterprise-level organization

INTERVIEW POINTS FROM THE NOTES

- Tenant Root Group exists by default.
- A custom logical hierarchy can be created with Terraform.
- Management Group Contributor and Directory Reader are listed in the notes for MG-level operations.
- Subscription association is handled with azurerm_management_group_subscription_association.
- Policies applied at higher MG levels have wider impact through inheritance.
- Child MGs can be created dynamically with for_each.
- Conditional creation can use count = var.enable_mg ? 1 : 0.
- MG ID can be referenced as azurerm_management_group.<name>.id.
- Azure Landing Zone discussions use MGs as a foundation for identity, management, connectivity and security.

DYNAMIC HIERARCHY PATTERN

for_each = var.management_groups
parent_management_group_id = each.value.parent

SUBSCRIPTION MAPPING

```
The notes propose a variable-driven subscription map and:
resource "azurerm_management_group_subscription_association" "sub" {
  for_each = var.subscription_map
}
```

POLICY ASSIGNMENT

The notes propose assigning policy at the production MG scope so child subscriptions inherit it.

EXISTING MG IMPORT

```
terraform import azurerm_management_group.m1 /providers/Microsoft.Management/managementGroups/<mg_id>
```

VERSIONING

For multi-repo MG modules, the notes recommend semantic versioning and pinned versions for production.

5. Docker — Multistage Build + Daily Commands

MULTISTAGE DOCKERFILE FROM THE NOTES

```
FROM node:18 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm build

FROM nginx:latest
COPY --from=builder /app/build /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

CONCEPT

Builder stage:
- Node image
- install dependencies
- copy source
- create production build

Runtime stage:
- Nginx
- copy only /app/build
- expose port 80

BUILD

```
docker build -t react-app:v1 .
```

LIST IMAGES

```
docker images
```

REMOVE IMAGES

```
docker image prune
docker image prune -a
```

```
The notes also contain:
docker rmi $(docker images -q)
```

```
RUN CONTAINER
```

```
docker run -d -p 8000:80 --name Asku node:v1
```

STOP

```
docker stop <container-id>
```

REMOVE STOPPED CONTAINERS

```
docker container prune
```

REMOVE ALL CONTAINERS

```
docker container rm $(docker ps -aq)
```

CLEANUP SUMMARY

```
docker container prune  -> stopped containers
docker image prune -a   -> unused images
docker system prune -a  -> broad cleanup
```

NO-CACHE BUILD

```
docker build --no-cache -t addtask-app .
```

The source notes specifically mention using --no-cache when an old Docker cache is still being used after a Dockerfile update.

6. Git — Remote, Push, Large Files, Secrets & Nested Repositories

REMOTE

```
git remote -v
```

CHANGE EXISTING ORIGIN

```
git remote set-url origin https://github.com/<user>/<repo>.git
```

ADD NEW ORIGIN

```
git remote add origin https://github.com/<user>/<repo>.git
```

BRANCH

```
git branch -M main
```

PUSH

```
git push -u origin main
```

LARGE FILE ISSUE

The spreadsheet records a GitHub rejection because a Terraform AzureRM provider executable was over GitHub's 100 MB file limit.

The practical root cause in the notes is that .terraform/provider binaries were being tracked.

The notes show git-filter-repo installation:
pip install git-filter-repo

```
and history cleanup using:
git filter-repo --path <path> --invert-paths --force
```

NESTED REPOSITORY

The notes show:
error: 'Enviroment/dev/' does not have a commit checked out
fatal: adding files failed

The documented diagnosis is a nested .git repository.

Practice:
cd Enviroment/dev
ls -Force
If .git exists and the directory is not intended to be a separate repository:

Remove-Item -Recurse -Force .git

```
Then return to the parent repository:
git add .
git status
```

LF / CRLF

The notes show:
LF will be replaced by CRLF

and explain this as a Windows line-ending warning.

```
Suggested configuration in the notes:
git config --global core.autocrlf true
```

SECRET SCANNING

The source notes contain a GitHub push rejected by secret scanning after a client_secret was committed.

Documented safe sequence:
1. Remove secret from source.
2. Rotate/revoke the exposed credential.
3. Remove it from history when necessary.
4. Add sensitive files to .gitignore.
5. Use environment variables / CI secrets / Key Vault.

The notes explicitly warn against treating GitHub's unblock-secret flow as the preferred security solution.

7. AKS + ACR + Kubernetes Practice

AKS / ACR FLOW FROM THE NOTES

```
1. Install Azure CLI on the VM where required.
2. az login
3. az acr login --name <ACR>
4. Attach ACR to AKS:
   az aks update --name <AKS-NAME> --resource-group <RG> --attach-acr <ACR>
5. Get AKS credentials:
   az aks get-credentials --name <AKS-NAME> --resource-group <RG>
6. Install kubectl if required:
   sudo az aks install-cli
7. Check:
   kubectl version --client
```

IMAGE PUSH

```
docker tag stream2:latest <ACR>.azurecr.io/stream2:latest
docker push <ACR>.azurecr.io/stream2:latest
az acr repository list --name <ACR> --output table
```

KUBECTL DOCUMENTATION

```
kubectl explain pod --recursive > pod.txt
kubectl explain deployment --recursive > deployment.txt
```

CREATE VS APPLY

```
kubectl create -f deployment.yaml
```

The notes describe create as a first-time resource creation command and note that it can error if the resource already exists.

```
kubectl apply -f deployment.yaml
```

The notes describe apply as create + update behavior and as useful for CI/CD.

ROLLOUT

```
kubectl rollout restart deployment <deployment-name>
```

IMAGE PULL PROBLEMS

ErrImagePull:
Initial image pull failed.

Possible reasons listed in the notes:
- wrong image name
- wrong tag

- private registry access
- network issue

ImagePullBackOff:
Repeated pull attempts continue to fail and Kubernetes enters a backoff state.

```
Troubleshooting:
kubectl describe pod <pod-name>
kubectl logs <pod-name>
```

For a private registry, check the registry identity/access path, image name/tag and connectivity.

8. Key Vault + VM / Linux Troubleshooting

The K8 issue notes contain Key Vault data-source examples for VM and database credentials.

The original notes contained real-looking usernames, passwords, an API key and an ODBC connection string. Those credential values are intentionally redacted in this consolidated book.

KEY VAULT DATA SOURCE PATTERN

```
data "azurerm_key_vault" "kv" {
  name                = "<key-vault-name>"
  resource_group_name = "<resource-group-name>"
}

data "azurerm_key_vault_secret" "example" {
  name         = "<secret-name>"
  key_vault_id = data.azurerm_key_vault.kv.id
}
```

VM / LINUX ISSUE — APT NOT WORKING

```
Step 1:
ip a
ip route

Step 2 — test internet without DNS:
ping -c 4 8.8.8.8

Step 3 — inspect DNS:
cat /etc/resolv.conf

The notes show DNS examples including:
nameserver 168.63.129.16
nameserver 8.8.8.8

systemd-resolved:
systemctl status systemd-resolved
```

The notes also show a temporary /etc/resolv.conf edit with public DNS values.

HOSTNAME ISSUE

```
hostname
cat /etc/hosts
```

Expected pattern from the notes:
127.0.0.1 localhost
127.0.1.1 <hostname>

```
Then:
sudo apt update
sudo apt upgrade -y
```

NGINX / SITE CANNOT BE REACHED

```
Check:
sudo systemctl status nginx

Check port:
sudo ss -tuln | grep 80

Test locally:
curl http://<VM-private-IP>
```

The notes say that if local access works, investigate Azure firewall / NSG access.

9. Terraform Interview Scenario Bank

FOUNDATION

1. Remote backend / state locking
2. Environment separation
3. Drift detection
4. Hardcoded credentials

NETWORKING & SECURITY

5. NSG misconfiguration
6. Private DB
7. CIDR overlap

SCALING & HIGH AVAILABILITY

8. Single VM to HA
- VMSS
- Availability Zones
- Load Balancer health probe

9. Zero downtime update
- VMSS rolling upgrade
- Blue-Green
- backend pool swap

10. Auto scaling

PRODUCTION SAFETY

11. Accidental destroy
12. Resource deletion protection
13. Plan approval workflow

STATE & RECOVERY

14. State corruption
- Blob version restore
- Import

15. State lock stuck

COST

16. High Azure bill
- right sizing
- auto shutdown
- reserved capacity

17. Mandatory tagging

ENTERPRISE

18. Multi-region DR
19. Import existing infrastructure
20. Modular design
- inputs
- outputs
- version control

SENIOR BONUS

21. Breaking change in module
22. Dependency failure / depends_on
23. Parallel apply issue

INTERVIEW ANSWER PATTERN

Scenario → identify root cause → check actual Azure/Kubernetes/Git state → make the smallest safe change → verify with plan/status/describe → document prevention.

10. Daily One-Page Revision

```
AZURE
az login
az account list -o table
az account show --query id -o tsv
az account set --subscription <id>
az group show --name <rg>
az vm list -o table

TERRAFORM
terraform init
terraform validate
terraform plan
terraform apply
terraform state list
terraform import <address> <id>
terraform state rm <address>
terraform destroy

GIT
git status
git remote -v
git branch
git add .
git commit -m "<message>"
git push -u origin main
git remote set-url origin <url>

DOCKER
docker build -t app:v1 .
docker images
docker run -d -p 8000:80 --name app app:v1
docker stop <id>
docker container prune
docker image prune -a
docker system prune -a
docker build --no-cache -t app .

AKS
az acr login --name <acr>
az aks update --name <aks> --resource-group <rg> --attach-acr <acr>
az aks get-credentials --name <aks> --resource-group <rg>
kubectl get pods
kubectl describe pod <pod>
kubectl logs <pod>
kubectl apply -f deployment.yaml
kubectl rollout restart deployment <deployment>
```

TROUBLESHOOTING ORDER

Azure:
CLI → resource exists? → permissions → network → Terraform state → plan

Terraform:
error → resource address → Azure actual state → state → config → plan

Git:
status → remote → branch → .gitignore → secrets/large files → commit → push

Docker:
Dockerfile → build → image → container → port → logs

```
AKS:
kubectl get → describe → logs → image/tag → ACR access → network
```

11. Azure DevOps SPN + WIF — Existing Portal Practice

The uploaded portal practice presentation documents this flow:

AKS-SPN
↓
Client ID + Tenant ID
↓
ADO Azure Resource Manager Service Connection
↓

Workload Identity Federation
↓
Issuer + Subject
↓
Federated Credential
↓
Subscription RBAC
↓
Verify
↓
AzureCLI az account show
↓
Terraform
↓
AKS

ADO side:
Project Settings → Service connections → New service connection → Azure Resource Manager

WIF side:
Issuer + Subject are copied from the ADO service connection and added to the App Registration's Federated credentials.

Federated credential:
Audience = api://AzureADTokenExchange

RBAC:
Subscription → Access control (IAM) → Add role assignment → Contributor → service principal.

```
Validation:
AzureCLI task using the service connection and:
az account show
```

Terraform sequence:
fmt → init → validate → plan → apply

AKS:
Resource Group → VNet/Subnet → ACR → Log Analytics → AKS → kubectl → workloads.